

Uncertain Commuters Assignment Through Genetic Programming Hyper-Heuristic

Xiao-Cheng Liao^{id}, Ya-Hui Jia, *Member, IEEE*, Xiao-Min Hu^{id}, *Member, IEEE*,
and Wei-Neng Chen^{id}, *Senior Member, IEEE*

Abstract—Traffic assignment problem (TAP) is of great significance for promoting the development of smart city and society. It usually focuses on the deterministic or predictable traffic demand and the vehicle traffic assignment. However, in the real world, traffic demand is usually unpredictable, especially the foot traffic assignment inside buildings such as shopping malls and subway stations. In this work, we consider the dynamic version of TAP, where uncertain commuters keep entering the traffic network constantly. These dynamically arriving commuters bring new challenges to this problem where planning paths for each commuter in advance is incompetent. To address this problem, we propose a genetic programming (GP) hyper-heuristic method to assign uncertain commuters in real-time. Specifically, a low-level heuristic rule called reactive assignment strategy (RAS) is proposed and is evolved by the proposed method. All commuters obey the same strategy to route themselves based on their local observations in a traffic network. Through training based on a designed heuristic template, all commuters will have the ability to find their appropriate paths in real-time to maximize the throughput of the traffic network. This decentralized control mechanism can address dynamically arriving commuters more efficiently than centralized control mechanisms. The experimental results show that our method significantly outperforms the state-of-the-art methods and the evolved RAS has a certain generalization ability.

Index Terms—Genetic programming (GP), routing, traffic assignment.

I. INTRODUCTION

PUBLIC transportation serves as an indispensable pillar of the social system [1]. However, with the global trend of urbanization and the rapid growth of population, unprecedented traffic demands are emerging [2], [3], [4]. Merely relying on improving transportation facilities, such as expanding or building roads, can no longer meet the growing public transportation demands [5]. Therefore, it is very important to carry out demand management on the basis of the existing transportation facilities.

Manuscript received 20 October 2022; revised 11 January 2023 and 27 February 2023; accepted 29 March 2023. This work was supported in part by the National Natural Science Foundation of China under Grant 61976093, Grant 62206098, and Grant 62272108; and in part by the Guangdong Regional Joint Foundation Key Project under Grant 2022B1515120076. (Corresponding authors: Ya-Hui Jia; Wei-Neng Chen.)

Xiao-Cheng Liao and Wei-Neng Chen are with the School of Computer Science and Engineering, South China University of Technology, Guangzhou 510006, China, and also with the State Key Laboratory of Subtropical Building Science, South China University of Technology, Guangzhou 510006, China (e-mail: cschenwn@scut.edu.cn).

Ya-Hui Jia is with the School of Future Technology, South China University of Technology, Guangzhou 510006, China, and also with Pazhou Laboratory, Guangzhou 510005, China (e-mail: jiayahui@scut.edu.cn).

Xiao-Min Hu is with the School of Computer Science and Technology, Guangdong University of Technology, Guangzhou 510006, China.

Digital Object Identifier 10.1109/TCSS.2023.3265727

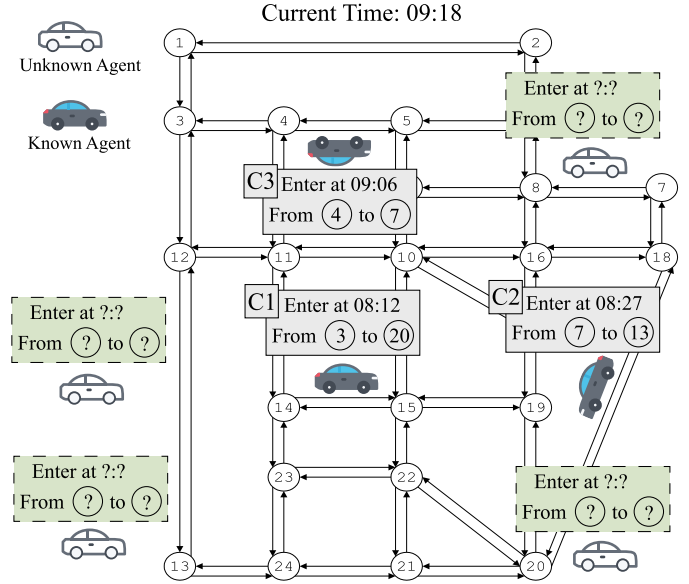


Fig. 1. Illustration of the UATAP problem. The current time is 9:18. C1 is a(n) commuter/agent destined for node 20, entered the road network from node 3 at 8:12, and is currently traveling on road (14,15). C2 is an agent destined for node 13, entered the road network from node 7 at 8:27, and is currently traveling on road (18,20). C3 is an agent destined for node 7, just entered the road network from node 4 at 9:06, and is currently traveling on road (4,5). In real-world scenarios, there are always agents (i.e., white car in this figure) that we cannot infer when and where they will enter the road network and their destinations. The goal of this problem is to maximize the throughput of the traffic network.

One of the most important problems of transportation planning is the traffic assignment problem (TAP). It is to allocate suitable traffic flows to the links in a road network for a given travel demand, usually represented by agents (commuters can be regarded as agents in this article) with different origin–destination (OD) pairs. TAP is a very challenging problem that many methods have been proposed to address it, including mathematical programming [6], [7], [8], [9], [10], heuristic algorithms [11], [12], [13], [14], [15], [16], [17], and metaheuristic algorithms [18], [19]. In addition to these methods, reinforcement learning also becomes a popular way to solve the TAP in the past few years [20], [21], [22].

So far, most work have focused on studying the TAP in a static environment, in which the time that all the agents entering the transportation network are known in advance [10], [23]. Only a few studies have tried to analyze the traffic assignment efficiency with dynamically arriving agents [24], [25], [26]. In all these studies, the delay function is known in advance and the agents are assumed to arrive sequentially in several fixed epochs [27]. However, in a real-

world road network, it is difficult for us to get the specific formula of the delay function. Furthermore, in reality, the environment is highly dynamic and the information of new agents is completely unknown until the agents enter the network. For example, Fig. 1 shows a traffic scenario with the dynamic and uncertain traffic demand. Considering the ubiquitous uncertainty in reality, we will take TAP as a black box model and focus on addressing the uncertain agent TAP (UATAP) in this article.

Compared with TAP, UATAP is even more challenging. First, classic TAP optimization methods that generate deterministic solutions, such as mathematical algorithms and metaheuristic algorithms, are unable to effectively handle the dynamic environment. When a new agent enters the road network, the preplanned solution needs to be adjusted to reroute the new agent and the remaining agents. Due to the high computational cost of these methods [18], [19], [20], [21], [22], re-optimizing the routes efficiently for all the agents in real-time is almost impossible. Second, no agent flow information can be inferred from the preplanned routes. When building a solution, many studies such as [14], [15], [16], [17] calculate the agent density of each link based on the traffic volume that has been assigned to the link. Through this analytical way, these methods can solve the TAP without traffic simulation. However, in UATAP, we need to simulate the uncertainty of new emerging agents, meaning that the algorithms should be evaluated based on real-time traffic simulations since new agents constantly enter the transportation network. Third, a little congestion in somewhere the transportation network may cause more serious congestion of the new agents, making the problem intractable. Finally, the TAP has a very large search space [20] that all the agents can choose any path to their destinations without any restrictions. The constantly arriving new agents will make the search space of the problem larger and larger.

In this article, we propose a reactive assignment strategy (RAS) to address UATAP with dynamic agents. RAS is used in the execution process of traffic simulation, to assign the traffic into the adjacent links at each node of the transportation network. When an agent arrives at a node, based on the analysis of the information surrounding the node, RAS can decide the most suitable target node for the agent to move forward. In this way, route planning for all the agents is deferred to the execution process instead of the preparing process. Since RAS makes decisions based on the surrounding information of nodes, the path planning for the unassigned agents can take the real-time states of the network into consideration. Essentially, RAS is a function that takes the surrounding information as input and outputs the next node to go to. It takes very little time to execute. Thus, it is very suitable to use RAS to solve UATAP.

However, designing an effective RAS is difficult, due to the following reasons. First, various factors related to the target agent, the decision node, and the adjacent links of the node should be taken into consideration when designing an RAS. Second, the designed RAS is adopted by all the agents, and its subtle differences may lead to a dramatic change in the overall performance. Third, designing an effective RAS manually

requires domain knowledge and high expertise. Moreover, RAS heavily depends on the traffic model, and different RASs need to be designed for different traffic models. Therefore, the automatic design of RAS is very important.

Genetic programming (GP) [28] is a famous algorithm to automatically generate expressions of arbitrary complexity. It has been used as a hyper-heuristic technique, named GPHH, to solve many dynamic problems, such as dynamic vehicle routing and dynamic job shop scheduling [29], [30]. In this work, we use it to find the (near-)optimal RAS for UATAP. To the best of our knowledge, no GPHH method has been designed for evolving RAS for UATAP.

To achieve the goal of designing effective RAS for UATAP automatically, we make the following contributions in this article. First, an UATAP model is formulated and investigated. The proposed model takes the dynamic agents into consideration where nothing of an agent can be inferred in advance until they enter the traffic network. In other words, the traffic demand can only be known at runtime.

Second, we proposed a GPHH algorithm to solve UATAP.

- 1) An effective RAS is proposed to route agents in the traffic network online.
- 2) A terminal set including features of the problem for RAS is designed and a famous variant of GP called gene expression programming (GEP) is used to evolve the RAS.
- 3) A heuristic template is created that can generate a feasible solution for a given RAS and an UATAP instance.

Finally, to test different algorithms, a new UATAP benchmark set is designed. This benchmark set contains a wide range of scenarios that can be used to generate various UATAP instances. Moreover, the benchmark set contains both the car traffic model and crowd traffic model. Five different volume-delay functions (VDFs) are used in the benchmark set. A few representative traffic networks including two real-world road networks are considered. The experimental results demonstrate that the proposed method is effective and efficient.

The rest of this article is organized as follows. Section II introduces the background including traffic assignment and GPHH. Section III formulates the proposed UATAP. The proposed method is demonstrated in detail in Section IV. Section V conducts the experiments and analyzes the results. Finally, Section VI draws the conclusion.

II. RELATED WORK

A. Traffic Assignment

TAP is frequently studied in the field of urban traffic planning. Traffic assignment is to allocate suitable traffic flows to the links in a road network for a given travel demand [23]. According to the traffic delay pattern in the transportation network, traffic assignment can be broadly classified into static traffic assignment (STA) and dynamic traffic assignment (DTA).

1) *Static Traffic Assignment:* STA regards the traffic flow velocity as a time-independent constant value [31], whereas the traffic flow and its velocity in DTA vary with respect to time [23]. Static traffic models fail to capture the dynamic

characteristics of the traffic flow [32], but due to their high computational efficiency and simplicity, they are still widely preferred tools in many strategic transportation planning problems [33], [34], [35].

2) *Dynamic Traffic Assignment*: In reality, traffic delay and traffic congestion are ubiquitous in the transportation network [36]. Sometimes, a little increase in agents' density on the road would cause a sharp decrease in the average traveling velocity of agents. The static traffic models cannot capture the essential features of traffic delay and traffic congestion, making it difficult to be applied in real life. DTA adapts dynamic traffic models that are able to effectively describe the time-varying changes in traffic flow. Since DTA is closer to real-world traffic scenarios, many researchers have paid significant attention to it in the past decade [18], [19], [20]. The research on DTA can be generally summarized into two categories: 1) analytical method and 2) simulation-based method.

Analytical Methods for DTA: The analytical methods handle DTA by solving analytical formulations through optimal control theory, mathematical programming, and variational inequalities [37]. When the delay function on road network links is integrable, TAP is usually decomposed into linearized subproblems and is converted into a convex mathematical program [38], [39]. When the delay function is non-integrable, TAP can also be formulated as a variational inequality problem [40] or a nonlinear complementarity problem [41], [42]. Therefore, it can be addressed by mathematical methods [6], [7], [8], [9], [10], [43], [44], [45] such as the classical Frank–Wolfe method [46]. These analytical methods have received a lot of attention due to their simplicity and scalability in large transportation networks. However, their inherent simplicity may lead to the lack of ability to deal with complex real-world traffic problems [47]. The validity and applicability of these methods rest heavily on the stability of their components [48]. Ignoring the spatial-temporal interactions of traffic flows may yield solutions that are far from correct [49]. This deviation is unacceptable in a practical sense.

Simulation-Based Methods for DTA: The simulation-based methods replicate the complex traffic flow dynamics by various traffic simulators. Since there are presently no analytical models of traffic flow that successfully replicate traffic theoretic relationships and provide well-behaved mathematical formulations [50], the simulation-based methods have been widely accepted in traffic assignment research due to their great reliability in relation to realistic traffic models. In the past decade, many simulation-based DTA methods have been proposed. For examples, Han et al. [49] studied the emergency management on a real-world scenario based on the simulation-based DTA model. Tian et al. [51] introduced a variable time discretization strategy for a time-dependent shortest path algorithm to address DTA. Shou et al. [20] proposed a multiagent reinforcement learning algorithm to address DTA, in which each commuter is modeled as a reinforcement learning agent.

The above methods can address the TAP well and obtain excellent performance. However, they only consider that the involved agents are fixed or predictable [52], [53], [54]

during the execution process in some way [55]. When the environment is highly dynamic, these methods are unable to handle the unpredictable agent arrivals during the execution process and may perform inefficient.

3) *Dynamic Traffic Demand*: There is only a few work that studied dynamic traffic demand. Traditional methods addressing DTA with dynamic traffic demand are to adjust the preplanned solution based on the online data [55], [56], [57], [58]. Harks et al. [27] mathematically analyzed the assignment efficiency of sequentially arriving agents. Waller and Ziliaskopoulos [59] presented a chance constraint programming method to solve the DTA with dynamic demand. In their works, the traffic demand is presented as a set of random variables with predefined probability distributions and the problem is studied merely on a single scenario.

Most of the aforementioned studies on dynamic traffic demand are analytical methods which treat DTA in a discrete manner [37]. However, the mathematical formulations and constraints are continuously defined, which may cause conflict between the discrete model and real situations. In these methods, the traffic demand is usually modeled through a uniform distribution bounded in a demand matrix. In addition, when robustness is taken into account, solutions of these methods tend to satisfy the worst case [60]. Thus, their performance is limited in managing different road networks. Therefore, the research on simulation-based DTA with uncertain demand still requires further developments. Furthermore, many aforementioned studies on TAP have only focused on some specific car traffic scenarios or crowd traffic scenarios, which may be hard to transfer to other scenarios. In this work, we will consider different traffic models with different VDFs to propose a more general method.

B. Genetic Programming Hyper-Heuristic

Hyper-heuristic is a promising method that aims at addressing a variety of real-world combinatorial optimization problems such as timetabling, vehicle routing, and scheduling problem. A hyper-heuristic algorithm solves a problem by working in the heuristic space to find the best heuristic combinations instead of finding a good solution of the problem directly. When a hyper-heuristic algorithm uses GP to generate low-level heuristics, it is called GP hyper-heuristic (GPHH). GP is a population-based metaheuristic proposed by Koza [28]. Each individual in GP is a syntax tree that can represent a function or a computer program [61]. Due to its flexible genotype representation, it can evolve computer programs of arbitrary complexity automatically. In the field of generative hyper-heuristic, GP has become the most popular method recently [62]. For examples, Zhang et al. [63], [64] presented a multiobjective GP to evolve scheduling rules for dynamic flexible job-shop scheduling. Wang et al. [65] adapted GP to address the uncertain capacitated arc routing problem. Lin et al. [66] proposed a GPHH approach with repair-based decoding scheme to address the multiskill resource-constrained project scheduling problem. Gao et al. [67] proposed a reactive coordination strategy that is evolved by

GPHH to solve the dynamic multipoint dynamic aggregation problem.

III. PROBLEM DEFINITION

A transportation system consists of demand and supply. The supply means the road network that can be modeled as a directed graph $G(V, E)$, where $V = \{v_1, v_2, \dots, v_N\}$ is the set of nodes (i.e., sites or intersections) and E is the set of links/edges (i.e., roads). Each link $(u, v) \in E$ indicates a one-way road from site/intersection u to site/intersection v . It can be represented as a tuple $(u, v, f_{u,v}, C_{u,v}, t_0, \text{VDF})$, where $f_{u,v}$ is the flow of agents occupying the road (u, v) , $C_{u,v}$ is the capacity of road (u, v) , t_0 is the free-flow travel time on (u, v) , and VDF is the VDF which abstracts the travel time of crossing a road. It maps $f_{u,v}$, $C_{u,v}$, and t_0 to an actual travel time t . The relationship between the actual travel time t and an example VDF can be represented as follows:

$$t = \text{VDF} \left(1 + a \left(\frac{f_{u,v}}{C_{u,v}} \right)^b \right) t_0 \quad (1)$$

where both a and b are variables defined for the physical characteristics of the traffic road network.

Another part of the transportation system is the demand. In TAP, the traffic demand is usually represented by an OD matrix. In UATAP, we can infer nothing until an agent appears at its origin. That is, when and where the agents enter the traffic network are totally unknown. Each agent $\mathcal{A}_k$ is modeled by a tuple with six elements $\mathcal{A}_k = (v_o^k, v_d^k, v_c^k, t_o^k, t_d^k, \mathcal{P}_k)$. v_o^k and v_d^k are the origin node and the destination node of the agent, respectively. t_o^k and t_d^k are the time that $\mathcal{A}_k$ enters the traffic network and the time that $\mathcal{A}_k$ leaves the traffic network, respectively. v_c^k is the current target node of $\mathcal{A}_k$ in the traffic network, i.e., the next node $\mathcal{A}_k$ should go to. $\mathcal{P}_k$ records the path that $\mathcal{A}_k$ has passed. When an agent $\mathcal{A}_k$ arrives at its destination v_d^k , its travel path can be represented as $\mathcal{P}_k = \{(v_o^k, v_a), \dots, (v_i, v_j), \dots, (v_b, v_d^k)\}$. t_d^k can be calculated by the following equation:

$$t_d^k = t_o^k + \sum_{(u,v) \in \mathcal{P}_k} t_{u,v} \quad (2)$$

where $t_{u,v}$ is the travel time of $\mathcal{A}_k$ on road (u, v) , which can be estimated by (1).

In an open transportation network, steady streams of agents flow into the transportation network from different origins to different destinations. They may enter the network at any time and leave after reaching their destinations. In this case, we expect as many agents as possible to pass through the network in the same amount of time. Therefore, the throughput of the transportation network becomes a very important indicator. The objective of our work is to maximize the throughput of the transportation network by assigning agents appropriately

$$\mathbb{F}(\mathcal{A}) = |\{\mathcal{A}_k \in \mathcal{A} \wedge t_d^k < T\}|. \quad (3)$$

In other words, given a fixed period of time $[0, T]$ and a set of agents $\mathcal{A} = \{\mathcal{A}_1, \mathcal{A}_2, \dots\}$, we aim to maximize the

number of agents that have finished their journey, arrived at their destinations, and left the transportation network

$$\max \mathbb{F}(\mathcal{A}). \quad (4)$$

IV. GENETIC PROGRAMMING HYPER-HEURISTIC FOR UATAP

This section describes the newly proposed GPHH, which evolves RASs for UATAP. In this section, the heuristic template that takes an RAS to generate a traffic assignment solution for a given UATAP instance is presented first, followed by the description of the evolved RAS. Then, the overall framework of the evolutionary process is presented.

A. Heuristic Template

Our method is a generation constructive hyper-heuristic [62] that searches in the heuristic space rather than the problem solution space. The trained low-level heuristic is essentially a priority function, which ranks the solution components. The heuristic template is used to simulate the component selection process to generate a solution.

Given an UATAP instance $\mathbb{H}$ and an RAS $\Gamma(\cdot)$, a solution of UATAP is a set of paths of all the involved agents/commuters. In this work, we plan path for each agent online. Therefore, only when each agent $\mathcal{A}_k$ arrived at its destination v_d^k can we know its full path $\mathcal{P}_k$. The designed heuristic template in this article is presented in Algorithm 1.

At the initialization stage (lines 1–3), the simulation time t is set to zero. Since there are no agents in the transportation network, we set $\mathcal{A}$ empty. To simplify the implementation details, we introduce two queues to assist the description of the proposed heuristic template, namely, the initial agent queue Q_0 and the active agent queue Q . Before the simulation starts, both of them are set empty.

When the simulation starts, the simulation process at each time t can be decomposed into two stages: 1) agents' initialization stage for handling new agents and 2) agents' decision-making stage for handling active agents.

At the agents' initialization stage (lines 5–13), we first collect all the agents that enter the network at time t and add them into queue Q_0 . The information of each new agent $\mathcal{A}_k$ we can know are only its origin node v_o^k and destination node v_d^k . Then, we treat each agent $\mathcal{A}_k$ in the queue Q_0 as follows. Since each agent $\mathcal{A}_k$ enters the traffic network at time t and has not moved, we set the enter time t_o^k to the current simulation time t and the travel path $\mathcal{P}_k$ to an empty set (lines 7–8). We assign the target node v_c^k of each agent $\mathcal{A}_k$ to its origin node v_o^k (lines 9). This assignment facilitates the recording path $\mathcal{P}_k$ of $\mathcal{A}_k$ during the agents' decision-making stage. This assignment is also reasonable. The agents newly entering the traffic network are usually from other traffic networks. In this case, before entering the current traffic network, the target node v_c^k of an agent $\mathcal{A}_k$ must be its origin node v_o^k . Finally, we remove all the agents in initial agents' queue Q_0 and add them to the agents' set $\mathcal{A}$ (lines 10–11).

At the decision-making stage (lines 14–29), we first collect all the agents that have arrived at their target nodes into the active agent queue Q . Then, two situations are distinguished.

Algorithm 1 Low-Level Heuristic Template

Input: an UATAP instance $\mathbb{H}$, an RAS $\Gamma(\cdot)$
Output: the fitness value $\mathbb{F}(\mathcal{A})$

```

1 Agents set  $\mathcal{A} \leftarrow \emptyset$ , simulation time  $t \leftarrow 0$ ;
2 Initial agents queue  $Q_0 \leftarrow \emptyset$ ;
3 Active agents queue  $Q \leftarrow \emptyset$ ;
4 while  $t < T$  do
5   Detect and collect agents that enter the traffic
   network at time  $t$  into  $Q_0$ ;
6   for each agent  $\mathcal{A}_k$  in  $Q_0$  do
7      $t_o^k \leftarrow t$ ;
8      $\mathcal{P}_k \leftarrow \emptyset$ ;
9      $v_c^k \leftarrow v_o^k$ ;
10     $\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathcal{A}_k\}$ ;
11     $Q_0 \leftarrow Q_0 \setminus \{\mathcal{A}_k\}$ ;
12  end
13  Detect and collect any agent  $\mathcal{A}_k$  that arrives its target
  node  $v_c^k$  into  $Q$ ;
14  for each agent  $\mathcal{A}_k$  in  $Q$  do
15    if  $v_c^k == v_d^k$  then
16       $t_d^k \leftarrow t$ ;
17       $Q \leftarrow Q \setminus \{\mathcal{A}_k\}$ ;
18    else
19      Get the neighbors set  $V_c$  of  $v_c^k$  by  $\mathbb{H}$ ;
20      for each  $v_i$  in  $V_c$  do
21        Get the status features  $\chi_i$  of  $v_i$  by  $\mathbb{H}$  at
        time  $t$ ;
22        Calculate the priority value  $p_{v_i} \leftarrow \Gamma(\chi_i)$ ;
23      end
24       $v_c^{k*} \leftarrow \arg \max_{v_i \in V_c} p_{v_i}$ ;
25       $\mathcal{P}_k \leftarrow \mathcal{P}_k \cup \{(v_c^k, v_c^{k*})\}$ ;
26       $v_c^k \leftarrow v_c^{k*}$ ;
27       $Q \leftarrow Q \setminus \{\mathcal{A}_k\}$ ;
28    end
29  end
30   $t \leftarrow t + 1$ ;
31 end
32 Calculate  $\mathbb{F}(\mathcal{A})$  by (3) and return it;

```

The first situation (lines 15–17) is that agent $\mathcal{A}_k$ has already arrived at its destination, i.e., $v_d^k = v_c^k$. In this situation, we just need to set t_d^k of $\mathcal{A}_k$ to the current time t and remove $\mathcal{A}_k$ from the active agent queue Q .

The second situation (lines 18–27) is that agent $\mathcal{A}_k$ has not arrived at its destination yet, i.e., $v_d^k \neq v_c^k$. In this situation, we need to plan the next node for $\mathcal{A}_k$. In other words, we need to update v_c^k of $\mathcal{A}_k$. First, the neighbor node set V_c of v_c^k is obtained from the instance $\mathbb{H}$. We require not only the neighbor nodes but also the real-time status features χ_i of each node v_i in V_c . Then, the given RAS takes the status features χ_i as input and outputs the priority value p_{v_i} of the neighbor node v_i . Given an agent $\mathcal{A}_k$ at v_c^k , its next target node v_c^{k*} is the neighbor v_i of v_c^k that has the highest priority among all the neighbors. Finally, we add (v_c^k, v_c^{k*}) to the path $\mathcal{P}_k$ of $\mathcal{A}_k$,

update the value of v_c^k , and remove $\mathcal{A}_k$ from the active agents' queue Q .

The agents' initialization stage and the agents' decision-making stage will run repeatedly until the maximum simulation time T is reached. After the simulation is finished, the number of agents that have finished their journeys can be calculated by (3). Therefore, given an UATAP instance $\mathbb{H}$ and an RAS $\Gamma(\cdot)$, we are able to measure the performance of $\Gamma(\cdot)$ on $\mathbb{H}$ based on the designed template.

B. RAS Representation

In our work, RAS is exactly the low-level heuristic we need to optimize. It is a tree-based priority function that can decide the most suitable target node v_c^k for a given agent $\mathcal{A}_k$ to move forward. An RAS comprises operators and attributes. The attributes of the problem are supposed to represent all the aspects of the problem domain as much as possible. However, we cannot include too many attributes because it may lead to an exceedingly large heuristic space, which may further result in high training and processing time, and it would also be hard to find a suitable heuristic [62]. When an agent $\mathcal{A}_k$ arrives at a node v_c^k , the RAS is used to calculate the priority value for each neighbor v_i of node v_c^k .

In this work, there are four attributes associated with v_i considered in RAS, i.e., χ_i contains four elements

$$\chi_i = \{L_\chi, S_\chi, \rho_\chi, \hat{\rho}_\chi\} \quad (5)$$

where L_χ is the length of the link (v_c^k, v_i) and S_χ is the length of the shortest path between node v_i and the destination node v_d^k of the agent $\mathcal{A}_k$. They are both attributes implying the geographic distance [68] that is significant in traffic assignment. ρ_χ is the normalized density on the link (v_c^k, v_i) that can be calculated by the following equation:

$$\rho_\chi = \frac{f_{v_c^k, v_i}}{C_{v_c^k, v_i}} \quad (6)$$

$\hat{\rho}_\chi$ is the average density of links of v_i to all its neighbors

$$\hat{\rho}_\chi = \frac{1}{|V'_c|} \frac{\sum_{u_i \in V'_c} f_{v_i, u_i}}{\sum_{u_i \in V'_c} C_{v_i, u_i}} \quad (7)$$

where V'_c is a set containing all the neighbors of the node v_i .

Besides the four variables aforementioned in (5), to enhance the expression ability of RAS, the first ten prime numbers are also added to the RAS attributes (i.e., the terminal set of GP) [28]. Thus, all the attributes of RAS are as follows:

$$\{2, 3, 5, 7, 11, 13, 17, 19, 23, 29\} \cup \{L_\chi, S_\chi, \rho_\chi, \hat{\rho}_\chi\}. \quad (8)$$

To ensure the final RAS being simple, the operators of RAS only contain four elementary arithmetic operations including addition $+$, subtraction $-$, multiplication $\times$, and division $\div$.

Fig. 2 shows an example RAS with the tree representation. This tree can be transformed into a mathematical function

$$\Gamma(X_i) = \frac{L_\chi}{\rho_\chi + \frac{3}{7}\hat{\rho}_\chi} + S_\chi. \quad (9)$$

According to this function, the priority value of each neighbor of v_c^k can be calculated. Therefore, agents at v_c^k are able to route themselves based on the priority values.

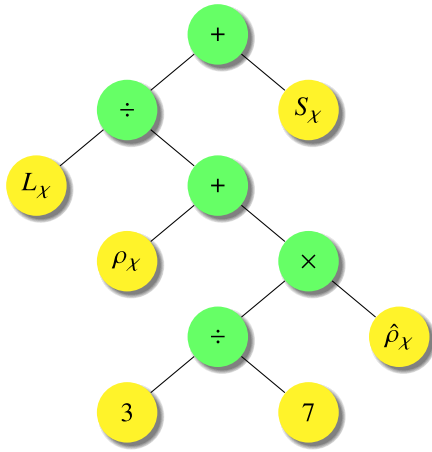


Fig. 2. Tree representation of an RAS example. The green nodes of the tree are operators of the RAS. The yellow nodes of the tree are attributes of the RAS.

RAS only needs the local information of the current node v_c^k . Since it does not rely on the specific VDF formulation, we can regard UATAP as a black-box optimization problem. This is exceedingly important because it is almost impossible to get a specific traffic pattern in the real world.

C. Evolution Process

Designing RAS manually based on human intuition is a time-consuming and laborious process that requires high expertise. Automating this process can decrease the cost of involved man-hours of developing low-level heuristics and may result in new constructive heuristics that human would not think of. In this work, we use GEP to generate the optimal RAS that combines or configures the attributes, constants, and operators.

GEP is a famous variant of GP, proposed by Ferreira [69]. GEP is distinguished by its unique gene expression representation. It models solutions by fixed length symbolic sequences instead of the traditional tree structure. With the help of the fixed-length gene expression representation, GEP can produce more succinct and readable solutions than the traditional GP.

Since the aforementioned heuristic template can measure the performance of a given RAS $\Gamma(\cdot)$ on a instance $\mathbb{H}$, we can use the heuristic template as the evaluation function

$$\text{fitness} = \text{HeuristicTemplate}(\Gamma, \mathbb{H}). \quad (10)$$

Thus, the goal of GEP is to find an optimal arithmetic function Γ to maximize the number of agents that have finished their journeys.

The overall framework of GPHH is presented in Algorithm 2. First, a population containing a set of individuals is randomly generated. Each individual in the population represents a candidate RAS Γ^* . Then, each individual is evaluated by (10) based on the simulator. After evaluating all the individuals, we apply genetic operators including: 1) selection; 2) point mutation; 3) inversion mutation; 4) insertion sequence (IS) mutation; 5) root IS (RIS) mutation; 6) one-point crossover; and 7) two-point crossover to generate a new population. These operations are all standard operations that can be found in [69]. The evolutionary process continues

Algorithm 2 High-Level Algorithm

Input: A UATAP instance $\mathbb{H}$
Output: A near-optimal RAS Γ

```

1 Population  $\leftarrow$  init(), current  $\leftarrow$  0;
2 while current < max_iteration do
3   Calculate fitness value of each candidate  $\Gamma^*$  by (10);
4   Offspring  $\leftarrow$  Selection(Population);
5   Offspring  $\leftarrow$  PointMutation(Offspring);
6   Offspring  $\leftarrow$  InversionMutation(Offspring);
7   Offspring  $\leftarrow$  ISMutation(Offspring);
8   Offspring  $\leftarrow$  RISMutation(Offspring);
9   Offspring  $\leftarrow$  PointsCrossover(Offspring);
10  Offspring  $\leftarrow$  TwoPointsCrossover(Offspring);
11  Population  $\leftarrow$  Offspring;
12  current  $\leftarrow$  current + 1;
13 end
14  $\Gamma \leftarrow \arg \max_{\Gamma^* \in \text{Population}} \text{HeuristicTemplate}(\Gamma^*, \mathbb{H})$ ;
15 return  $\Gamma$ ;
```

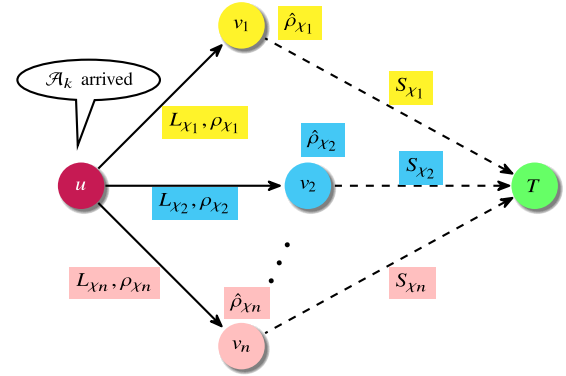


Fig. 3. Example that the agent $\mathcal{A}_k$ reaches node u . The green node is the destination of agent $\mathcal{A}_k$. Nodes with labels v_i (i.e., $v_1, v_2, \dots, v_n$) are neighbors of u .

until the stopping criterion is met, and the best individual of the final population is returned as the final RAS $\Gamma(\cdot)$.

D. RAS Application Example

We present an example in Fig. 3 to explain how RAS is applied in the UATAP problem. In a road network, commuters/agents often pass through many intersections, that is, the nodes in the road network. The proposed RAS is actually a routing rule to dynamically navigate agents at each node. As shown in Fig. 3, the agent $\mathcal{A}_k$ has just reached node u , and thus $\mathcal{A}_k$ faces a problem that which neighbor of the node u should it go to. In this case, the RAS Γ can take the state information of each neighbor v_i of u as input and output a priority value for each neighbor v_i . Therefore, by (11) we can calculate the next node u' that $\mathcal{A}_k$ should go to

$$u' = \arg \max_{v_i \in \{v_1, v_2, \dots, v_n\}} p_{v_i} \quad (11)$$

$$p_{v_i} = \Gamma(\chi_i) = \Gamma(L_{\chi_i}, S_{\chi_i}, \rho_{\chi_i}, \hat{\rho}_{\chi_i}).$$

Then, when $\mathcal{A}_k$ arrives at the node u' , its next node can be calculated in the same way based on neighbors of u' . When

$\mathcal{A}_k$ continues to reach its next node, do the same until it reaches its destination. Once the target RAS is trained, we can embed it into a modern navigation software to navigate commuters at intersections.

E. Time Complexity

Referring to Fig. 3, L_{χ_i} and S_{χ_i} are the basic properties of the road network that hardly change and can be obtained in a constant time. ρ_{χ_i} and $\hat{\rho}_{\chi_i}$ are the agent density of road (u, v_i) and surrounding road of v_i , which are dynamically changing over time. Thanks to the popularity of road surveillance cameras and the development of internet of vehicles technology [70], the calculation of these two variables is potentially close to constant time. Each time an agent is routed, RAS will be calculated several times, depending on the number of out-degrees of node u . From Table I in Section V-B, we can know that the average number of out-degree is very small in real-world road networks. Therefore, the calculation of next node for each agent can be regarded as being able to complete in constant time. Thus, the time complexity of the proposed algorithm depends on the total number K of agents involved and the average number $\mathcal{L}$ of nodes traveled by agents. $\mathcal{L}$ is indeterminate and contingent upon the performance of the applied RAS. To simplify the analysis, we consider the upper bound of the algorithm, $\mathcal{L} = N$. Therefore, the time complexity of the proposed algorithm is $O(NK)$. In real world, the road network size typically remains constant and $N \ll K$, and thus the execution time of the algorithm is primarily influenced by K . The above is based on the theoretical analysis, and the specific execution time can refer to the experimental results in Section V-F.

V. EXPERIMENTS

In this section, we set up the experiment in the first place including setting the parameters of the algorithm and generating the testing benchmark set. Then, the proposed method is compared with some representative algorithms, and the experimental results are analyzed in detail.

A. GEP Setting

We set the population size $NP = 32$ and the maximum number of iterations as 100, based on the recommendations from the previous work [69], [71] and our preliminary experiments. The probabilities of different GEP operators are carefully set as follows. Point-mutation rate, inversion-mutation rate, IS-mutation rate, and RIS-mutation rate are set to 0.1. The probabilities of one-point crossover rate and two-point crossover rate are 0.6 and 0.4, respectively.

In addition, all the experiments are implemented in Python, run on a PC with four Intel Core i7-10700F 2.90-GHz CPU, 8-GB memory.

B. Scenarios

As aforementioned, there is no existing benchmark set that contains uncertain agents currently. To facilitate the study of UATAP, a new benchmark set with a wide range of scenarios

is designed. These instances differ from each other from three aspects, **traffic network topology, uncertain traffic demand, and VDF.**

1) *Road Network Topology*: The benchmark adopts both the **synthetic networks and real-world networks**. The synthetic networks include four types of artificial networks: Regular network [termed as $RG(N, k)$], Watts–Strogatz small-world network [termed as $WS(N, k, p)$], grid network [termed as $GRID(dim_1, dim_2)$], and triangular lattice network [termed as $TRI(dim_1, dim_2)$], where N is the number of nodes in the network, k is the average degree of generated networks, p is the probability in generating links, and dim_i is the i th dimension size of a geometric network.

Besides, three well-known real-world networks are adopted in our work. The first network is the famous Sioux Falls network (SF) [72] containing 24 nodes and 76 links, which has been used in many publications to test the traffic assignment algorithms. The second one is the Anaheim network (ANH) [73] containing 416 nodes and 914 links. The third one is the Eastern Massachusetts (EMA) [74] containing 74 nodes and 258 links. These datasets contain some basic information about roads including length, capacity, and free flow time, which are the necessary data to build simulations. The road capacity of these road networks is usually relatively large, leading to that the number of commuters on the road has an insignificant effect on the travel time. Thus, we scale all the road capacities down to make the test scenarios more challenging. In addition, we assume that all the roads in these road networks are **homogeneous**, that is, the relationship between the road travel time and commuter density is uniform.

2) *Traffic Demand*: Since there is no existing benchmark set that contains uncertain traffic demands, we designed two methods to generate OD pairs (ODs) set S_{ods} and uncertain arrivals, respectively.

The first method is to generate challenging ODs. Our basic idea is to take the distinct endpoints of the $\lfloor N/5 \rfloor$ longest path as ODs. First, we calculate all the shortest paths set for all the connected nodes' pairs of the traffic network by Dijkstra's algorithm and term it S_{path} . Second, we sort S_{path} by the length in descending order. Then, we sequentially take each path from S_{path} with its endpoint pair (u, v) . If u or v is already in OD pairs set S_{ods} , we ignore this pair of endpoints. Otherwise, we add this (u, v) to S_{ods} . When the size of S_{ods} is equal to $\lfloor N/5 \rfloor$, the algorithm ends. The detailed process of this method is presented in Algorithm 3. To make our scenarios even more challenging, at the beginning of the simulation, the number of agents that enter the road network from all the origins of S_{ods} increases rapidly to one-fifth of the total network capacity.

At the same time, we designed a method to generate uncertain arrivals. Once an agent reaches its destination, after a random period of time, a new agent will be generated at a random origin to a random destination. Under this mechanism, the traffic demand is unknown in advance and difficult to predict.

3) *Volume-Delay Function*: In TAP, especially the **foot traffic assignment**, traffic patterns differ greatly in different traffic networks. In this work, five different VDFs are

Algorithm 3 Method of Generating ODs**Input:** a directed graph G , the number of OD pairs N_{ods} **Output:** OD pairs set S_{ods}

```

1 Results set  $S_{ods} \leftarrow \emptyset$ , loop counter  $cur \leftarrow 0$ ;
2 Calculate all the shortest paths set  $S_{path}$  for all the
  connected nodes' pairs of  $G$  by Dijkstra's algorithm in
   $O(N^2 \log N)$  time;
3 Sort paths in  $S_{path}$  in descending order by the path
  length;
4 for each path  $J_{path}$  in  $S_{path}$  do
5   Get the endpoint pair  $(u, v)$  from  $S_{path}$ ;
6   if  $u$  in  $S_{ods}$  or  $v$  in  $S_{ods}$  then
7     Continue
8   end
9   if  $cur \& 1$  then
10     $S_{ods} \leftarrow S_{ods} \cup (u, v)$ 
11  else
12     $S_{ods} \leftarrow S_{ods} \cup (v, u)$ 
13  end
14  if  $cur \geq N_{ods}$  then
15    Break out of the loop
16  end
17   $cur \leftarrow cur + 1$ ;
18 end

```

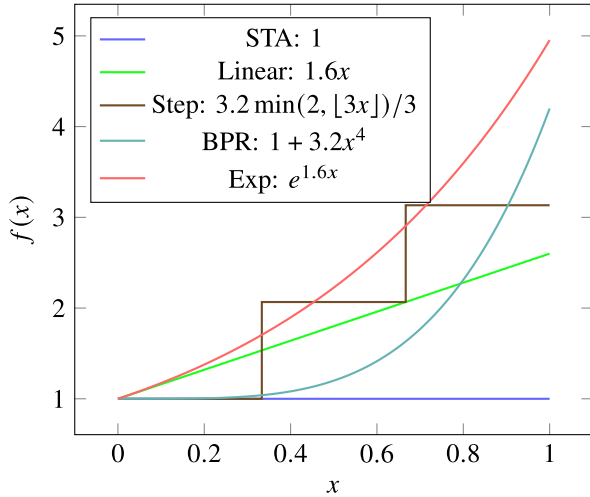


Fig. 4. Five VDFs adopted in our work.

adopted in the designed benchmark set, including constant function (STA), linear function (Linear), step function (Step), exponential function (Exp), and the widely used *Bureau of Public Roads* function (BPR) [75]. Their curves and expressions are presented in Fig. 4. These five VDFs can reflect most of the common traffic patterns in our real life.

In summary, we designed 52 scenarios based on the above combination of road network topology and VDFs. Each scenario combining with a given uncertain traffic demand becomes an UATAP instance. We named these scenarios by their road network topology and VDFs. For example, a scenario name is WS_1000_BPR, indicating a WS network that has 1000 nodes and adapts BPR as its VDF. In this

TABLE I
STATISTICS FOR OUT-DEGREE OF ROAD NETWORKS

	Min	Mean	Median	Max	Std
SF	2	3.17	3	5	0.75
Anaheim	1	2.2	2	6	1.18
EMA	1	3.49	4	12	1.82
WS_1000	2	4	4	8	0.91
RG_1000	4	4	4	4	0

Min, Mean, Median, Max and *Std* represent the minimum, average, median, maximum and standard deviation of the out-degree numbers of nodes, respectively

TABLE II
RAS TRAINING AND TESTING SCENARIOS OF THE
WS NETWORK AND THE RG NETWORK

Train Scenario	Test Scenario
WS_100_[VDF]*	WS_100_[VDF]
	WS_200_[VDF]
	WS_500_[VDF]
	WS_1000_[VDF]
RG_100_[VDF]*	RG_100_[VDF]
	RG_200_[VDF]
	RG_500_[VDF]
	RG_1000_[VDF]

For scenarios of both WS network and RG network, we train the RAS from one instance and test it on other unseen instances of larger scenarios. VDF can be any one of the VDFs adopted in this work, including STA, Linear, Step, BPR, and Exp.

work, all the WS networks are only different from each other in the network size and are termed $WS(N, 4, (1/4))$. The RG networks are also only different from each other in the network size and are termed $RG(N, 4)$. The grid networks and triangular lattice networks adopted in this work are $GRID(10, 10)$ and $TRI(16, 16)$, respectively.

In the traffic network, the degree of nodes is a very important attribute. The statistics for out-degree of road networks are shown in Table I. We can see that the degree distribution of the synthetic network (e.g., WS_1000 and RG_1000) is very similar to that of the real road network. In classic urban road networks, intersections surrounded by four roads are the most common intersections. Therefore, we adopt the synthetic network with the average out-degree number of 4.

Except for the WS-network- and RG-network-based scenarios (names start with “WS_” or “RG_”), we train an RAS on a random instance and test it on 30 unseen instance for each scenario. As for the WS-network- and RG-network-based scenarios, we train the RASs on small networks (networks with 100 nodes) and apply the RASs to larger traffic networks. The detailed experimental setup of this is presented in Table II. For example, the RAS tested on instances of WS_200_BPR, WS_500_BPR, and WS_1000_BPR is trained from an instance of WS_100_BPR.

C. Comparison Design

In UATAP, agents appear dynamically somewhere in the network. The existing metaheuristic methods cannot be applied in this problem directly. Therefore, the following four RASs, including manually designed RASs and RASs adapted from other well-performed heuristic methods (i.e., capacity constrained route planner ++ (CCRP++) [15], capacity-aware

shortest path evacuation routing (CASPER) [17], and most urgent evacuee group first (MUEGF) [76]), are taken to compare. All the compared RASs are designed to cater to the characteristics of UATAP so that they are able to handle dynamical and uncertain agents.

Specifically, two manually designed RASs are adapted, i.e., the greedy algorithm [77] and Dijkstra's algorithm [78]. The greedy algorithm in our experiments focuses on the lengths of roads in the simulation, where each agent always selects the shortest road as its next road. Dijkstra's algorithm was proposed by Dijkstra for solving the shortest path problem, through which the shortest paths from one node to all the other nodes can be found. Dijkstra's algorithm in our experiments guides each agent to follow the shortest path between its OD. The CCRP++ algorithm is proposed to address the STA, and the CASPER is a heuristic approach to solve the DTA under the condition that the VDF of a given traffic network is known in advance. MUEGF is a heuristic method with a mechanism prioritizing agents to solve the foot TAP.

D. Experiment Results

The computational results of the five methods are shown in Table III. Each row presents the average number of finished agents on 30 different instances of the same scenario. For every compared method, the nonparametric Wilcoxon rank-sum test at a significance level of 0.05 is conducted with Bonferroni correction. Symbol $-$ means the compared method performs significantly worse than our method. Symbol $+$ represents the compared method performs significantly better than our method.

It can be seen in Table III that our method significantly outperforms all the other compared methods in most scenarios. Only on three scenarios with the constant function, WS_500_STA, WS_1000_STA, and GRID_STA, it was beaten by the compared algorithms. Generally, although the local information of an agent contains only several attributes, it is still challenging to design an effective RAS manually for a given UATAP instance. It is difficult to foresee the implications of routing agents based on their local information on the overall performance. In contrast, GEP has an excellent ability to evolve sophisticated and flexible RASs. The proposed GPHH algorithm can effectively learn the useful knowledge of UATAP that is hard to be identified by a human from the historical data.

As for the three scenarios in which GPHH was beaten, the VDFs of them are all the constant function. Therefore, the travel time of agents on any link of the transportation network is fixed. The travel time on a link is only related to the length of the link and has nothing to do with the agent density on the link. The problem in these three scenarios turns into the STA problem. Redundant density-dependent features of RAS weaken its performance. The performance of GPHH is slightly weaker than CCRP++ and MUEGF in few STA scenarios but outperforms all the compared methods in other scenarios.

E. Feature Analysis

We choose the RASs trained from the WS network of 100 nodes with different VDFs as an example. Fig. 5 shows

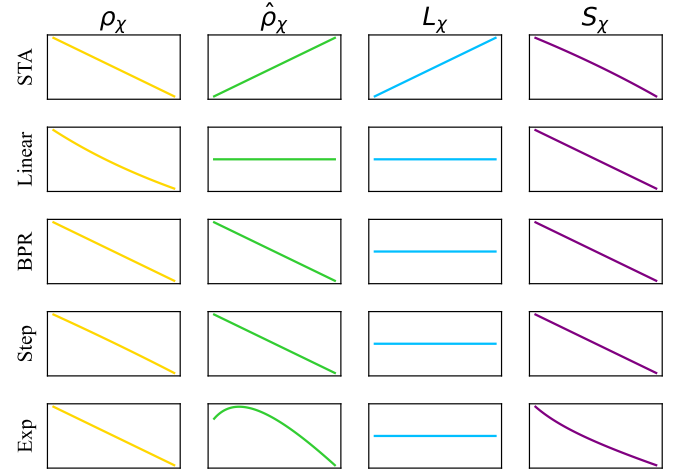


Fig. 5. Feature analysis.

the effect of changes in a single dimension on the overall value of the function when other dimensions are fixed.

From Fig. 5, the following observations can be obtained.

- 1) ρ_χ and S_χ are two significant features that represent the agent density on the target road and the distance between the target road and the destination. This two features show the same trend in all the RASs of the figure, reflecting that ρ_χ and S_χ are two exceedingly important features for the UATAP problem. In all the RASs of the figure, neighbors v_i of v_c^k with smaller ρ_χ and S_χ have higher priority for agent routing.
- 2) $\hat{\rho}_\chi$ reflects the congestion level of the next node. It can be seen that most RASs tend to prefer v_i with smaller $\hat{\rho}_\chi$.
- 3) The feature L_χ is not frequently used by these RASs. We infer that the difference in length of each road in the transportation network is not particularly significant. Compared with the distance from v_c^k to its neighbor v_i , the distance from v_i to the destination v_d^k usually is more important. Another reasonable explanation is that in WS networks, the lengths of different links do not differ very much. For two links with similar lengths, if the VDF is not STA, the traveling time of an agent on these links would be greatly influenced by the density instead of the length.

The detailed expressions of these evolved RASs are presented in Table IV. Although they have very similar correlations with the four variables, we can see that each RAS has its unique form to make itself specific to the training scenario. We can also see that all the RASs are relatively simple rules without some complex structures such as power function structure. A reasonable explanation is that the target problem requires a relatively simple RAS to obtain a good global performance. A complex RAS may cause it to perform very well somewhere but perform bad at other places in the road network.

F. Convergence Analysis

The convergence curves of the training process of our method on SF_BPR and Anaheim_BPR are presented in

TABLE III
COMPARISON RESULTS WITH DIFFERENT ALGORITHMS

Columns represent different algorithms and rows represent different scenarios.

Each value in the same row represents the average number of finished agents, for each compared algorithm, on 30 different instances of the corresponding scenario.

Sig. is the Wilcoxon rank-sum test results with Bonferroni correction, in which notation "+" indicates that GEPHH performs significantly better than the corresponding algorithm, notation "-" indicates that GEPHH performs significantly worse than the corresponding algorithm and notation "≈" indicates that GEPHH has comparable performance to the corresponding algorithm.

For each scenario, the algorithm with the best performance is bolded. Our algorithm significantly outperforms other compared algorithms on 49 out of 52 scenarios. GEPHH performs significantly worse than the compared algorithms on only three scenarios with VDFs in the form of STA.

The last column presents the performance of the RAS trained from an instance of the scenario "WS_100_Step" and we name it GEPHH*. In this column, the black color represents GEPHH* performs better than other compared algorithms (except GEPHH), and the gray represents it does not.

ID	Scenario name	Greedy	Sig.	Dijkstra	Sig.	CCRP++	Sig.	CASPER	Sig.	MUEGF	Sig.	GEPHH	GEPHH*
1	WS_100_STA	2.792e+03	-	1.785e+03	-	1.599e+04	-	1.557e+04	-	1.557e+04	-	1.876e+04	1.795e+04
2	WS_100_Linear	2.349e+03	-	1.349e+03	-	7.997e+03	-	9.392e+03	-	9.147e+03	-	1.017e+04	1.014e+04
3	WS_100_BPR	2.047e+03	-	1.278e+03	-	6.681e+03	-	8.338e+03	-	8.336e+03	-	1.183e+04	9.696e+03
4	WS_100_Step	2.299e+03	-	1.317e+03	-	6.784e+03	-	7.171e+03	-	7.172e+03	-	8.990e+03	8.989e+03
5	WS_100_Exp	2.021e+03	-	1.263e+03	-	5.058e+03	-	6.422e+03	-	6.344e+03	-	7.475e+03	7.043e+03
6	WS_200_STA	1.516e+04	-	9.233e+03	-	3.903e+04	-	3.903e+04	-	4.128e+04	≈	4.134e+04	3.989e+04
7	WS_200_Linear	1.141e+04	-	6.048e+03	-	1.839e+04	-	2.007e+04	-	2.076e+04	-	2.396e+04	2.466e+04
8	WS_200_BPR	8.605e+03	-	3.680e+03	-	1.535e+04	-	2.148e+04	-	2.284e+04	-	2.701e+04	2.303e+04
9	WS_200_Step	1.026e+04	-	5.410e+03	-	1.605e+04	-	1.688e+04	-	1.746e+04	-	2.211e+04	2.212e+04
10	WS_200_Exp	7.563e+03	-	3.500e+03	-	1.179e+04	-	1.538e+04	-	1.553e+04	-	1.668e+04	1.712e+04
11	WS_500_STA	1.913e+04	-	1.463e+04	-	7.560e+04	+	7.560e+04	+	8.146e+04	+	7.198e+04	7.601e+04
12	WS_500_Linear	1.341e+04	-	1.047e+04	-	3.755e+04	-	4.120e+04	-	4.273e+04	-	4.535e+04	4.842e+04
13	WS_500_BPR	9.533e+03	-	8.503e+03	-	2.895e+04	-	3.457e+04	-	3.609e+04	-	4.877e+04	4.674e+04
14	WS_500_Step	1.185e+04	-	9.395e+03	-	3.240e+04	-	3.388e+04	-	3.468e+04	-	4.411e+04	4.412e+04
15	WS_500_Exp	8.729e+03	-	7.921e+03	-	2.364e+04	-	2.884e+04	-	2.879e+04	-	3.451e+04	3.489e+04
16	WS_1000_STA	3.979e+04	-	3.710e+04	-	1.472e+05	+	1.472e+05	+	1.507e+05	+	1.455e+05	1.469e+05
17	WS_1000_Linear	2.921e+04	-	2.636e+04	-	7.620e+04	-	8.330e+04	-	8.326e+04	-	9.218e+04	9.747e+04
18	WS_1000_BPR	2.230e+04	-	2.286e+04	-	6.229e+04	-	7.477e+04	-	7.356e+04	-	1.006e+05	9.763e+04
19	WS_1000_Step	2.707e+04	-	2.510e+04	-	6.593e+04	-	7.196e+04	-	7.066e+04	-	9.039e+04	9.039e+04
20	WS_1000_Exp	2.064e+04	-	2.188e+04	-	4.876e+04	-	6.051e+04	-	5.881e+04	-	7.276e+04	7.213e+04
21	RG_100_STA	1.166e+04	-	1.168e+04	-	2.952e+04	-	2.952e+04	-	2.734e+04	-	3.057e+04	2.844e+04
22	RG_100_Linear	5.555e+03	-	5.806e+03	-	1.510e+04	-	1.647e+04	-	1.633e+04	-	1.888e+04	1.794e+04
23	RG_100_BPR	3.872e+03	-	5.158e+03	-	1.286e+04	-	1.627e+04	-	1.709e+04	-	2.335e+04	1.731e+04
24	RG_100_Step	5.677e+03	-	5.586e+03	-	1.302e+04	-	1.340e+04	-	1.424e+04	-	1.626e+04	1.626e+04
25	RG_100_Exp	4.178e+03	-	4.602e+03	-	9.910e+03	-	1.249e+04	-	1.251e+04	-	1.540e+04	1.251e+04
26	RG_200_STA	1.448e+04	-	1.307e+04	-	4.914e+04	-	4.914e+04	-	5.072e+04	-	5.099e+04	5.283e+04
27	RG_200_Linear	9.935e+03	-	1.055e+04	-	2.471e+04	-	2.840e+04	-	2.931e+04	-	3.286e+04	3.349e+04
28	RG_200_BPR	7.528e+03	-	8.337e+03	-	2.126e+04	-	2.692e+04	-	2.922e+04	-	4.236e+04	3.438e+04
29	RG_200_Step	9.011e+03	-	9.756e+03	-	2.142e+04	-	2.467e+04	-	2.634e+04	-	2.879e+04	3.136e+04
30	RG_200_Exp	6.989e+03	-	7.671e+03	-	1.580e+04	-	2.165e+04	-	2.185e+04	-	2.776e+04	2.503e+04
31	RG_500_STA	3.513e+04	-	2.660e+04	-	1.092e+05	-	1.092e+05	-	1.149e+05	-	1.241e+05	1.184e+05
32	RG_500_Linear	2.424e+04	-	1.691e+04	-	5.589e+04	-	6.289e+04	-	6.483e+04	-	7.901e+04	7.503e+04
33	RG_500_BPR	1.859e+04	-	1.257e+04	-	4.482e+04	-	5.587e+04	-	5.959e+04	-	9.720e+04	7.576e+04
34	RG_500_Step	2.280e+04	-	1.403e+04	-	4.814e+04	-	5.433e+04	-	5.434e+04	-	6.758e+04	7.014e+04
35	RG_500_Exp	1.769e+04	-	1.234e+04	-	3.492e+04	-	4.729e+04	-	4.751e+04	-	6.476e+04	5.590e+04
36	RG_1000_STA	7.902e+04	-	4.939e+04	-	1.893e+05	-	1.893e+05	-	2.042e+05	-	2.113e+05	2.145e+05
37	RG_1000_Linear	5.360e+04	-	3.678e+04	-	9.737e+04	-	1.096e+05	-	1.191e+05	-	1.392e+05	1.399e+05
38	RG_1000_BPR	3.904e+04	-	2.910e+04	-	8.139e+04	-	9.969e+04	-	1.138e+05	-	1.684e+05	1.384e+05
39	RG_1000_Step	4.660e+04	-	3.275e+04	-	8.449e+04	-	9.612e+04	-	1.020e+05	-	1.201e+05	1.317e+05
40	RG_1000_Exp	3.593e+04	-	2.897e+04	-	6.277e+04	-	8.114e+04	-	8.767e+04	-	1.122e+05	1.037e+05
41	GRID_STA	4.969e+03	-	1.881e+03	-	1.265e+04	+	1.265e+04	+	1.241e+04	-	1.262e+04	1.028e+04
42	GRID_Linear	2.414e+03	-	1.514e+03	-	6.559e+03	-	7.077e+03	-	7.124e+03	-	7.160e+03	7.380e+03
43	GRID_BPR	1.185e+03	-	1.603e+03	-	6.361e+03	-	8.385e+03	-	8.712e+03	-	9.879e+03	9.505e+03
44	GRID_Step	2.116e+03	-	1.549e+03	-	5.825e+03	-	5.875e+03	-	6.080e+03	-	6.656e+03	7.142e+03
45	GRID_Exp	1.252e+03	-	1.297e+03	-	4.520e+03	-	5.316e+03	-	5.419e+03	-	6.170e+03	6.152e+03
46	TRI_STA	3.123e+03	-	1.817e+03	-	2.575e+04	-	2.575e+04	-	2.457e+04	-	2.622e+04	1.820e+04
47	TRI_Linear	2.224e+03	-	1.509e+03	-	1.215e+04	-	1.338e+04	-	1.342e+04	-	1.518e+04	1.397e+04
48	TRI_BPR	1.950e+03	-	1.076e+03	-	1.051e+04	-	1.466e+04	-	1.480e+04	-	1.974e+04	1.701e+04
49	TRI_Step	2.228e+03	-	1.379e+03	-	1.031e+04	-	1.125e+04	-	1.144e+04	-	1.411e+04	1.389e+04
50	TRI_Exp	1.944e+03	-	1.018e+03	-	7.336e+03	-	1.062e+04	-	1.070e+04	-	1.193e+04	1.179e+04
51	SF_BPR	1.553e+03	-	2.287e+03	-	6.632e+03	-	8.223e+03	-	8.735e+03	-	1.057e+04	8.773e+03
52	Anaheim_BPR	2.232e+03	-	1.328e+03	-	5.862e+03	-	7.826e+03	-	7.624e+03	-	1.103e+04	8.270e+03
53	EMA_BPR	1.803e+03	-	3.337e+03	-	1.117e+04	-	1.543e+04	-	1.469e+04	-	1.575e+04	1.457e+04

TABLE IV
EXAMPLES OF RAS EVOLVED FROM WS NETWORK WITH 100 NODES

Train Scenarios	RASs
WS_100_STA	$(x_0 - x_1 - x_2 + 42)/(17 * (x_3 - 7))$
WS_100_Linear	$(-31 * x_0 - 34 * x_3 * (x_0 + 3) + 40)/(374 * (x_0 + 3))$
WS_100_BPR	$-x_0/3 - x_1/21 - x_3/3 - 53/3$
WS_100_Step	$(6 * x_0 * x_3 + 3 * x_0 + 7 * x_1 + 17 * x_3 - 51)/(x_0 - 17)$
WS_100_Exp	$(17 * x_1 + 16 * x_3 - (x_1 + x_3) * (x_0 + x_1 + x_3 + 276))/(x_1 + x_3)$

where x_0, x_1, x_2, x_3 represent the terminals $L_X, S_X, \rho_X, \hat{\rho}_X$, respectively.

Fig. 6. The x -axis is the number of iterations, and the y -axis is the objective value. Due to the huge gap of fitness values among different solutions, we use the average fitness value of the best solution fitness value over ten repeated runs

in each generation to plot the curves. The 95% confidence interval for each convergence curve is shown as the shadowed regions, demonstrating the stability of the convergence of our algorithm. It can be seen in Fig. 6 that the objective values of both the instances gradually increase during the training process. Observing the shape of the confidence interval, we could see that although the deviation in the middle of the training process is large, finally they converged to similar values. Each confidence interval follows the corresponding average curve and fluctuates within a narrow range. Therefore, the convergence curves show that our method is stable to generate good RASs.

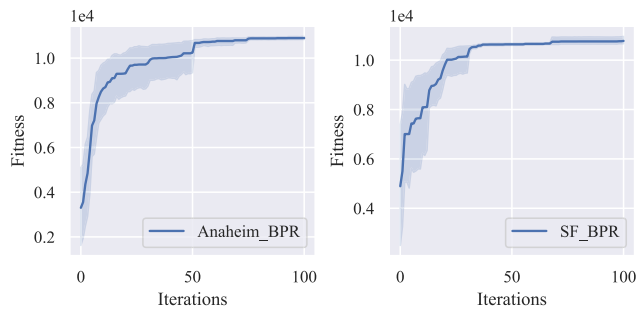


Fig. 6. Convergence curves of our algorithm on two real-world networks, that is, SF and Anaheim, respectively. The 95% confidence interval for each convergence curve is presented as the corresponding shadowed regions.

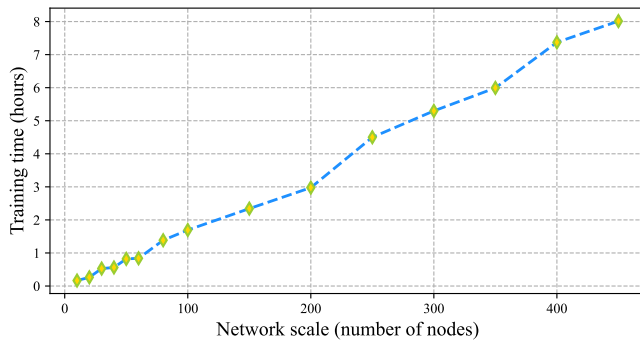


Fig. 7. Relationship between training time and network scale N in networks of $WS(N, 4, 0.25)$.

Fig. 7 shows the curves of the training time of our method across $WS(N, 4, 0.25)$ networks of different nodes. The x -axis represents the scales of transportation networks, and the y -axis is the training time measured by hours. Notably, the traffic network of more nodes is accompanied by more roads and greater traffic demand according to our experimental setting.

The training time of the proposed method approximately linearly increases as the scales of transportation networks and their accompanying traffic demands. The reason is that each routing for any agent can be finished in constant time since it only needs to perform a mathematical function calculation. Thus, the training time is linearly related to traffic demand. As the network scale increases, the OD distance of each agent increases, which also lead to an increase in the number of decisions for the agent and an increase in training time. However, the path length growth of each agent with respect to the increase in network scale is an extremely slow process. The relationship between the path length of an agent and the network scale is like the relationship between the area and the radius of a circle. Therefore, the increase in the transportation network scale has a very slight effect on the training time.

G. Generality Analysis

The proposed method has certain generalization ability in two aspects, including different traffic demands and different problem instances.

Referring to Table III, for each scenario, we train the RAS on only one instance and test it on 30 unseen instances with completely different traffic demands. That is to say, the RAS has certain generalization ability in terms of different traffic

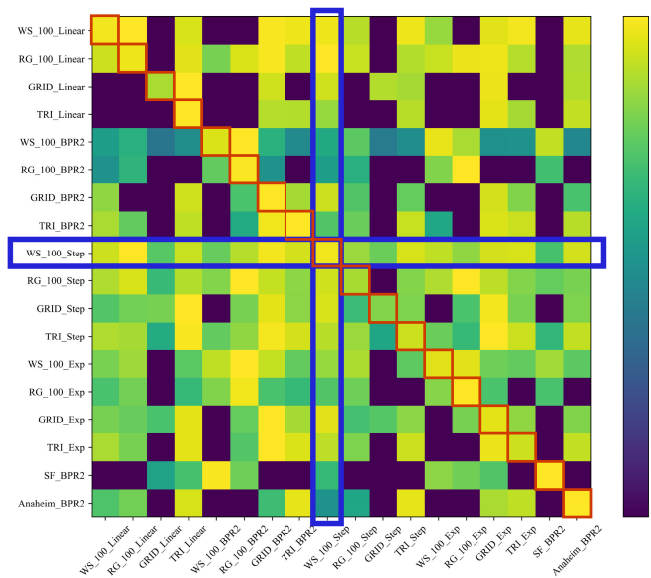


Fig. 8. Performance matrix of applying RASs trained from different scenarios to different scenarios. The x -dimension represents the RASs trained from different scenarios and the y -dimension represents the test scenarios.

demands. Regardless of the uncertain traffic demand, when given a problem instance with new traffic demand, many traditional methods such as metaheuristic [18] need to be restarted to produce a new solution. Moreover, we train the RAS on small network and test it on larger networks. The more detailed experimental designs can be found in Table III. The results in Table II demonstrate that the RAS trained on the small network instance (e.g., WS_{100_BPR}) can also perform well not only on other unseen instances of the same scenario (i.e., WS_{100_BPR}) but also on instances of scenarios with larger networks (e.g., WS_{200_BPR} , WS_{500_BPR} , and WS_{1000_BPR}). Therefore, the proposed RAS has some generalization ability in terms of scenarios with different scales of road networks.

In addition, we test the performance of the RAS trained from an instance of the scenario WS_{100_Step} , which is presented in the last column of Table III. In this column, the black color represents GEPHH* performs better than other compared algorithms (except GEPHH), and the gray represents it does not. We can see that the RAS trained from only one instance can perform very well in different scenarios in terms of different traffic networks and different traffic patterns (i.e., VDFs). Therefore, it is very potential to obtain an RAS with strong generalization ability through training it on scenarios with some general features.

We also apply the RASs trained on different scenarios to other scenarios to check their generalization performance. In this way, we calculate the average performance square matrix for ten repeated runs and present the results in Fig. 8. The x -dimension represents the RASs trained on different scenarios, and the y -dimension represents their performance on different test scenarios. Each purple area in the matrix represents that the corresponding case performs worse than CASPER. In other words, if a cell of the x th row and the y th column in the matrix is purple, indicating that the CASPER outperforms the RAS our GPHH trained on scenario x tested

on scenario y . In addition to purple areas, other areas represent the corresponding case performs better than CASPER. In these areas, the lighter the color, the better the performance, and vice versa.

It can be seen in Fig. 8 that in most cases, the trained RAS can have an excellent test performance and outperform the CASPER, demonstrating that the trained RAS on a single scenario also has a certain generalization ability for other scenarios. However, not many RASs can perform well in all the test scenarios. A rare example is the RAS trained on WS_100_Step (the blue column in Fig. 8). It is the only RAS that performs well on all the test scenarios. We can also see that all the trained RASs have a good performance on WS_100_Step (the blue row in Fig. 8). We infer that scenario WS_100_Step has some general features that can be used to train general RASs. First, the WS network topology has some general features that can better characterize road transportation network, genetic control networks, neural networks and many other self-organizing systems [ws]. Second, referring to Fig. 4, the step function has sharp rise from micro perspective and a steady rise from macro perspective. It incorporates multiple VDF features, demonstrating some general abilities. Nevertheless, scenarios with general features are extremely difficult to find.

In the real world, the road networks or scenarios are usually fixed that change infrequently. Only the traffic demand may change frequently. Thanks to the acceptable training time, we can train an RAS on a scenario instance and apply it on other instances of the same scenario. In this way, our method can always get an excellent performance on all the test scenarios. The red matrix diagonal in Fig. 8 proves this.

VI. CONCLUSION

The goal of this article was to solve UATAP effectively. This goal has been successfully achieved by proposing a GPHH method. Specifically, we have formally defined UATAP and proposed an RAS to handle the uncertain arrival of commuters. The experimental results showed that our method outperformed other compared algorithms in maximizing the transportation network throughput. Given any UATAP instance, with the help of the designed heuristic template, GEP can successfully evolve effective RAS that performs well in other instances with the same traffic network and the same traffic pattern (i.e., VDFs). Not only the specifically trained RAS is effective but also experiments showed that some RASs had great generalization ability. This is due to: 1) the approximately fixed traffic network and traffic pattern in the real world; 2) the acceptable training time of the RAS; and 3) the black-box treatment of the traffic pattern, and the proposed method is efficient and practical.

There are still some limitations in this work that we will consider in our future work. From the perspective of the problem model, in real world, agents may be pedestrians or cars that have completely different path selection preferences, and we want to take the heterogeneity of agents and nodes into consideration. From the perspective of the algorithm, currently the trained RAS only relies on some local information to

make decision, which may not be very effective in a global view. In the future, we will pay more attention to designing communication mechanisms in RAS.

REFERENCES

- [1] R. Verma et al., "A smartphone-based passenger assistant for public bus commute in developing countries," *IEEE Trans. Computat. Social Syst.*, vol. 7, no. 2, pp. 465–476, Apr. 2020.
- [2] U. Desa, *World Urbanization Prospects 2018*. New York City, NY, USA: United Nations Department for Economic and Social Affairs, 2018.
- [3] T. Wang, A. Hussain, L. Zhang, and C. Zhao, "Collaborative edge computing for social Internet of Vehicles to alleviate traffic congestion," *IEEE Trans. Computat. Social Syst.*, vol. 9, no. 1, pp. 184–196, Feb. 2022.
- [4] L. Cheng et al., "SCTSC: A semicentralized traffic signal control mode with attribute-based blockchain in IoVs," *IEEE Trans. Computat. Social Syst.*, vol. 6, no. 6, pp. 1373–1385, Dec. 2019.
- [5] H. Lu et al., "Social signal-driven knowledge automation: A focus on social transportation," *IEEE Trans. Computat. Social Syst.*, vol. 8, no. 3, pp. 737–753, Jun. 2021.
- [6] W. Vickrey, "Congestion theory and transport investment," *Amer. Econ. Rev.*, vol. 59, no. 2, pp. 251–260, Feb. 1969.
- [7] T. Akamatsu, K. Wada, T. Iryo, and S. Hayashi, "A new look at departure time choice equilibrium models with heterogeneous users," *Transp. Res. B, Methodol.*, vol. 148, pp. 152–182, Jun. 2021.
- [8] M. Osawa, H. Fu, and T. Akamatsu, "First-best dynamic assignment of commuters with endogenous heterogeneities in a corridor network," *Transp. Res. B, Methodol.*, vol. 117, pp. 811–831, Nov. 2018.
- [9] C.-Y. Li and H.-J. Huang, "Morning commute in a single-entry traffic corridor with early and late arrivals," *Transp. Res. B, Methodol.*, vol. 97, pp. 23–49, Mar. 2017.
- [10] T. Roughgarden and É. Tardos, "How bad is selfish routing?" *J. ACM*, vol. 49, no. 2, pp. 236–259, 2002.
- [11] M. Ameli, "Heuristic methods for calculating dynamic traffic assignment," Ph.D. thesis, IFSTTAR Université de Lyon, Lyon, France, 2019.
- [12] C. O. Tong and S. C. Wong, "Heuristic algorithms for simulation-based dynamic traffic assignment," *Transportmetrica*, vol. 6, no. 2, pp. 97–120, Apr. 2010.
- [13] Q. Lu, B. George, and S. Shekhar, "Capacity constrained routing algorithms for evacuation planning: A summary of results," in *Proc. Int. Symp. Spatial Temporal Databases*. Cham, Switzerland: Springer, 2005, pp. 291–307.
- [14] Q. Lu, Y. Huang, and S. Shekhar, "Evacuation planning: A capacity constrained routing approach," in *Proc. Int. Conf. Intell. Secur. Inform.* Cham, Switzerland: Springer, 2003, pp. 111–125.
- [15] D. Yin, "A scalable heuristic for evacuation planning in large road network," in *Proc. 2nd Int. Workshop Comput. Transp. Sci.*, Nov. 2009, pp. 19–24.
- [16] S. Kim, B. George, and S. Shekhar, "Evacuation route planning: Scalable heuristics," in *Proc. 15th Annu. ACM Int. Symp. Adv. Geographic Inf. Syst.*, Nov. 2007, pp. 1–8.
- [17] K. Shahabi and J. P. Wilson, "CASPER: Intelligent capacity-aware evacuation routing," *Comput., Environ. Urban Syst.*, vol. 46, pp. 12–24, Jul. 2014.
- [18] M. Ameli, J. Lebacque, and L. Leclercq, "Simulation-based dynamic traffic assignment: Meta-heuristic solution methods with parallel computing," *Comput.-Aided Civil Infrastruct. Eng.*, vol. 35, no. 10, pp. 1047–1062, Oct. 2020.
- [19] Z.-M. Huang, W.-N. Chen, Q. Li, X.-N. Luo, H.-Q. Yuan, and J. Zhang, "Ant colony evacuation planner: An ant colony system with incremental flow assignment for multipath crowd evacuation," *IEEE Trans. Cybern.*, vol. 51, no. 11, pp. 5559–5572, Nov. 2021.
- [20] Z. Shou, X. Chen, Y. Fu, and X. Di, "Multi-agent reinforcement learning for Markov routing games: A new modeling paradigm for dynamic traffic assignment," *Transp. Res. C, Emerg. Technol.*, vol. 137, Apr. 2022, Art. no. 103560.
- [21] R. Grunitzki and A. L. C. Bazzan, "Comparing two multiagent reinforcement learning approaches for the traffic assignment problem," in *Proc. Brazilian Conf. Intell. Syst. (BRACIS)*, Oct. 2017, pp. 139–144.
- [22] J. Sharma, P.-A. Andersen, O.-C. Granmo, and M. Goodwin, "Deep Q-learning with Q-matrix transfer learning for novel fire evacuation environment," *IEEE Trans. Syst., Man, Cybern. Syst.*, vol. 51, no. 12, pp. 7363–7381, Dec. 2021.

- [23] K. Saw, B. Katti, and G. Joshi, "Literature review of traffic assignment: Static and dynamic," *Int. J. Transp. Eng.*, vol. 2, no. 4, pp. 339–347, 2015.
- [24] T. Harks, "Nash equilibria in online sequential routing games," Zuse Inst. Berlin, Berlin, Germany, Tech. Rep. 06-43, 2006.
- [25] T. Harks and L. A. Végh, "Nonadaptive selfish routing with online demands," in *Proc. Workshop Combinat. Algorithmic Aspects Netw.* Cham, Switzerland: Springer, 2007, pp. 27–45.
- [26] T. Harks, S. Heinz, and M. E. Pfetsch, "Competitive online multicommodity routing," *Theory Comput. Syst.*, vol. 45, no. 3, pp. 533–554, Oct. 2009.
- [27] T. Harks, S. Heinz, M. Pfetsch, and T. Vredeveld, "Online multicommodity routing with time windows," Zuse Inst. Berlin, Berlin, Germany, Tech. Rep. 07-22, 2007.
- [28] J. R. Koza, "Genetic programming as a means for programming computers by natural selection," *Statist. Comput.*, vol. 4, no. 2, pp. 87–112, Jun. 1994.
- [29] B. Xu, Y. Mei, Y. Wang, Z. Ji, and M. Zhang, "Genetic programming with delayed routing for multiobjective dynamic flexible job shop scheduling," *Evol. Comput.*, vol. 29, no. 1, pp. 75–105, Mar. 2021.
- [30] F. T. Hanshar and B. M. Ombuki-Berman, "Dynamic vehicle routing using genetic algorithms," *Appl. Intell.*, vol. 27, no. 1, pp. 89–99, 2007.
- [31] R. Jayakrishnan, W. K. Tsai, and A. Chen, "A dynamic traffic assignment model with traffic-flow relationships," *Transp. Res. C, Emerg. Technol.*, vol. 3, no. 1, pp. 51–72, 1995.
- [32] M. J. Jihani, "A review of dynamic traffic assignment computer packages," *J. Transp. Res. Forum*, vol. 46, pp. 34–46, Oct. 2010.
- [33] D.-G. Zhang, J.-X. Wang, J. Zhang, T. Zhang, C. Yang, and K.-W. Jiang, "A new method of fuzzy multicriteria routing in vehicle ad hoc network," *IEEE Trans. Computat. Social Syst.*, early access, Aug. 1, 2022, doi: [10.1109/TCSS.2022.3193739](https://doi.org/10.1109/TCSS.2022.3193739).
- [34] G. Kim, Y. S. Ong, C. K. Heng, P. S. Tan, and N. A. Zhang, "City vehicle routing problem (City VRP): A review," *IEEE Trans. Intell. Transp. Syst.*, vol. 16, no. 4, pp. 1654–1666, Aug. 2015.
- [35] H. Huang, S. Yang, X. Li, and Z. Hao, "An embedded Hamiltonian graph-guided heuristic algorithm for two-echelon vehicle routing problem," *IEEE Trans. Cybern.*, vol. 52, no. 7, pp. 5695–5707, Jul. 2021.
- [36] Y. Pei and Y. Lang, "Research on dynamic traffic assignment applied in traffic congestion analyze and management," in *Proc. IEEE Int. Conf. Intell. Transp. Syst.*, vol. 2, Oct. 2003, pp. 1032–1035.
- [37] S. Peeta and A. K. Ziliaskopoulos, "Foundations of dynamic traffic assignment: The past, the present and the future," *Netw. Spatial Econ.*, vol. 1, no. 3, pp. 233–265, Sep. 2001.
- [38] S. C. Dafermos, "The traffic assignment problem for multiclass-user transportation networks," *Transp. Sci.*, vol. 6, no. 1, pp. 73–87, Feb. 1972.
- [39] M. Beckmann, C. B. McGuire, and C. B. Winsten, "Studies in the economics of transportation," Roy. Econ. Soc., Wiley, USA, Tech. Rep. 00201410, 1956.
- [40] M. J. Smith, "The existence, uniqueness and stability of traffic equilibria," *Transp. Res. B, Methodol.*, vol. 13, no. 4, pp. 295–304, 1979.
- [41] H. Z. Aashtiani, "The multi-modal traffic assignment problem," Ph.D. dissertation, Massachusetts Inst. Technol., Cambridge, MA, USA, 1979.
- [42] H. Z. Aashtiani and T. L. Magnanti, "Equilibria on a congested transportation network," *SIAM J. Algebr. Discrete Methods*, vol. 2, no. 3, pp. 213–226, 1981.
- [43] R. Lindsey, "Existence, uniqueness, and trip cost function properties of user equilibrium in the bottleneck model with multiple user classes," *Transp. Sci.*, vol. 38, no. 3, pp. 293–314, Aug. 2004.
- [44] Y. Nie and Y. F. Yin, "Managing rush hour travel choices with tradable credit scheme," *Transp. Res. B, Methodol.*, vol. 50, no. 4, pp. 1–19, Apr. 2013.
- [45] T. Iryo, "Properties of dynamic user equilibrium solution: Existence, uniqueness, stability, and robust solution methodology," *Transportmetrica B, Transp. Dyn.*, vol. 1, no. 1, pp. 52–67, Apr. 2013.
- [46] M. Frank and P. Wolfe, "An algorithm for quadratic programming," *Naval Res. Logistics Quart.*, vol. 3, nos. 1–2, pp. 95–110, 1956.
- [47] J. G. Sender and M. Netter, "Équilibre offre-demande et tarification SUR un réseau de transport, modèle ASTARTÉ (application de systèmes tarifaires à un réseau de transport: Trafics et tarifs d'Équilibre)," UC Berkeley Transp. Library, USA, Tech. Rep. 00839059, 1970.
- [48] T. Larsson and M. Patriksson, "An augmented Lagrangean dual algorithm for link capacity side constrained traffic assignment problems," *Transp. Res. B, Methodol.*, vol. 29, no. 6, pp. 433–455, Dec. 1995.
- [49] S. Han, S. Fang, X. Wang, X. Chen, and Y. Cai, "A simulation-based dynamic traffic assignment model for emergency management on the Hangzhou bay bridge," in *Proc. ICCTP*, Jul. 2010, pp. 883–895.
- [50] E. Mitsakis, J. M. Salanova, and G. Giannopoulos, "Combined dynamic traffic assignment and urban traffic control models," *Proc.-Social Behav. Sci.*, vol. 20, pp. 427–436, Jan. 2011.
- [51] Y. Tian, Y.-C. Chiu, and Y. Gao, "Variable time discretization for a time-dependent shortest path algorithm," in *Proc. 14th Int. IEEE Conf. Intell. Transp. Syst. (ITSC)*, Oct. 2011, pp. 588–593.
- [52] N. Sindlev Andersen, M. Chiarandini, and K. Debrabant, "A comparison of different approaches to dynamic origin-destination matrix estimation in urban traffic," 2022, *arXiv:2202.00099*.
- [53] T. Djukic, "Dynamic OD demand estimation and prediction for dynamic traffic management," Ph.D. thesis, Delft Univ. Technol., Delft, The Netherlands, 2014.
- [54] T. Toledo and T. Kolehkhina, "Estimation of dynamic origin-destination matrices using linear assignment matrix approximations," *IEEE Trans. Intell. Transp. Syst.*, vol. 14, no. 2, pp. 618–626, Jun. 2013.
- [55] S. Peeta and H. S. Mahmassani, "Multiple user classes real-time traffic assignment for online operations: A rolling horizon solution framework," *Transp. Res. C, Emerg. Technol.*, vol. 3, no. 2, pp. 83–98, Apr. 1995.
- [56] S. Peeta and C. Zhou, "Robustness of the off-line a priori stochastic dynamic traffic assignment solution for on-line operations," *Transp. Res. C, Emerg. Technol.*, vol. 7, no. 5, pp. 281–303, Oct. 1999.
- [57] Y.-C. Chiu and H. S. Mahmassani, "Toward hybrid dynamic traffic assignment-models and solution procedures," in *Proc. ITSC. IEEE Intell. Transp. Syst.*, Mar. 2001, pp. 245–250.
- [58] Y.-C. Chiu and H. S. Mahmassani, "Routing profile updating strategies for online hybrid dynamic traffic assignment operation," *Transp. Res. Rec., J. Transp. Res. Board*, vol. 1857, no. 1, pp. 39–47, Jan. 2003.
- [59] S. T. Waller and A. K. Ziliaskopoulos, "A chance-constrained based stochastic dynamic traffic assignment model: Analysis, formulation and solution algorithms," *Transp. Res. C, Emerg. Technol.*, vol. 14, no. 6, pp. 418–427, 2006.
- [60] G. C. Calafiore, M. Ghirardi, and A. Rizzo, "Robust dynamic traffic assignment for single destination networks under demand and capacity uncertainty," *J. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 331–351, Jul. 2020.
- [61] Y. Mei, Q. Chen, A. Lensen, B. Xue, and M. Zhang, "Explainable artificial intelligence by genetic programming: A survey," *IEEE Trans. Evol. Comput.*, early access, Nov. 28, 2022, doi: [10.1109/TEVC.2022.3225509](https://doi.org/10.1109/TEVC.2022.3225509).
- [62] N. Pillay and R. Qu, *Hyper-Heuristics: Theory and Applications*. Cham, Switzerland: Springer, 2018.
- [63] F. Zhang, Y. Mei, S. Nguyen, K. C. Tan, and M. Zhang, "Task relatedness based multitask genetic programming for dynamic flexible job shop scheduling," *IEEE Trans. Evol. Comput.*, early access, Aug. 18, 2022, doi: [10.1109/TEVC.2022.3199783](https://doi.org/10.1109/TEVC.2022.3199783).
- [64] F. Zhang, Y. Mei, S. Nguyen, and M. Zhang, "Multitask multiobjective genetic programming for automated scheduling heuristic learning in dynamic flexible job-shop scheduling," *IEEE Trans. Cybern.*, early access, Aug. 26, 2022, doi: [10.1109/TCYB.2022.3196887](https://doi.org/10.1109/TCYB.2022.3196887).
- [65] S. Wang, Y. Mei, and M. Zhang, "A multi-objective genetic programming algorithm with α dominance and archive for uncertain capacitated arc routing problem," *IEEE Trans. Evol. Comput.*, early access, Jul. 29, 2022, doi: [10.1109/TEVC.2022.3195165](https://doi.org/10.1109/TEVC.2022.3195165).
- [66] J. Lin, L. Zhu, and K. Gao, "A genetic programming hyper-heuristic approach for the multi-skill resource constrained project scheduling problem," *Exp. Syst. Appl.*, vol. 140, Feb. 2020, Art. no. 112915.
- [67] G. Gao, Y. Mei, B. Xin, Y.-H. Jia, and W. N. Browne, "Automated coordination strategy design using genetic programming for dynamic multipoint dynamic aggregation," *IEEE Trans. Cybern.*, vol. 52, no. 12, pp. 13521–13535, Dec. 2022.
- [68] Rimjhim, N. Cheke, J. Chandra, and S. K. Dandapat, "Understanding the impact of geographical distance on online discussions," *IEEE Trans. Computat. Social Syst.*, vol. 7, no. 4, pp. 858–872, Aug. 2020.
- [69] C. Ferreira, "Gene expression programming: A new adaptive algorithm for solving problems," 2001, *arXiv:cs/0102027*.
- [70] B. Ji et al., "Survey on the Internet of Vehicles: Network architectures and applications," *IEEE Commun. Standards Mag.*, vol. 4, no. 1, pp. 34–41, Mar. 2020.
- [71] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*. Reading, MA, USA: Addison-Wesley, 1989.

- [72] L. J. LeBlanc, E. K. Morlok, and W. P. Pierskalla, "An efficient approach to solving the road network equilibrium traffic assignment problem," *Transp. Res.*, vol. 9, no. 5, pp. 309–318, 1975.
- [73] Transportation Networks for Research Core Team. *Transportation Networks for Research*. (2022). [Online]. Available: <https://github.com/bstabler/TransportationNetworks>
- [74] J. Zhang, S. Pourazarm, C. G. Cassandras, and I. C. Paschalidis, "The price of anarchy in transportation networks: Data-driven evaluation and reduction strategies," *Proc. IEEE*, vol. 106, no. 4, pp. 538–553, Apr. 2018.
- [75] H. Spiess, "Conical volume-delay functions," *Transp. Sci.*, vol. 24, no. 2, pp. 153–158, 1990.
- [76] M. A. Dulebenets et al., "Exact and heuristic solution algorithms for efficient emergency evacuation in areas with vulnerable populations," *Int. J. Disaster Risk Reduction*, vol. 39, Oct. 2019, Art. no. 101114.
- [77] J. Edmonds, "Matroids and the greedy algorithm," *Math. Program.*, vol. 1, no. 1, pp. 127–136, 1971.
- [78] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numer. Math.*, vol. 1, no. 1, pp. 269–271, Dec. 1959.



Xiao-Min Hu (Member, IEEE) received the bachelor's and Ph.D. degrees from Sun Yat-sen University, Guangzhou, China, in 2006 and 2011, respectively.

She is currently an Associate Professor with the School of Computer Science and Technology, Guangdong University of Technology, Guangzhou. She has authored or coauthored more than 40 papers in international journals and conferences. Her current research interests include artificial intelligence, data mining, evolutionary computation, swarm intelligence, and their applications in design and optimization on wireless sensor networks, route optimization, and big data analysis.

Dr. Hu was a recipient of the Pearl River S&T Nova Program of Guangzhou in 2018.



Xiao-Cheng Liao received the bachelor's degree from the School of Information Science and Engineering, Yunnan University, Kunming, China, in 2020. He is currently pursuing the master's degree with the School of Computer Science and Engineering, South China University of Technology, Guangzhou, China.

His current research interests include evolutionary computation, multiagent systems, reinforcement learning, and their applications.



Ya-Hui Jia (Member, IEEE) received the B.Eng. and Eng.D. degrees from Sun Yat-sen University, Guangzhou, China, in 2013 and 2019, respectively.

He is currently an Assistant Professor with the School of Future Technology, South China University of Technology, Guangzhou. His current research interests include evolutionary computation, combinatorial optimization, software engineering, cloud computing, and intelligent transportation.



Wei-Neng Chen (Senior Member, IEEE) received the bachelor's and Ph.D. degrees in computer science from Sun Yat-sen University, Guangzhou, China, in 2006 and 2012, respectively.

Since 2016, he has been a Full Professor with the School of Computer Science and Engineering, South China University of Technology, Guangzhou. He has coauthored over 100 international journal and conference papers, including more than 60 papers published in the IEEE Transactions journals. His current research interests include computational intelligence, swarm intelligence, network science, and their applications.

Dr. Chen was a recipient of the IEEE Computational Intelligence Society (CIS) Outstanding Dissertation Award in 2016, and the National Science Fund for Excellent Young Scholars in 2016. He was also a Principle Investigator (PI) of the National Science and Technology Innovation 2030, the Next Generation Artificial Intelligence Key Project. He is currently the Vice-Chair of the IEEE Guangzhou Section, and the Chair of IEEE SMC Society Guangzhou Chapter. He is also a Committee Member of the IEEE CIS Emerging Topics Task Force. He serves as an Associate Editor for the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and the Complex and Intelligent Systems.